GSAP Layer Orchestration

Implementation plan — xmail "How it protects you" section

Target `src/components/site/LayerScene.tsx`, `src/components/site/EncryptionLayers.tsx` **Stack** React 18.3.1, Vite 7, @react-three/fiber, gsap 3.15.0, @gsap/react 2.1.2 **Status** Plan only. Nothing in this document has been implemented.

---

1. The problem, precisely

The section's thesis is stated in its own heading: "*Six layers, and you can look inside every one.*" The copy says the shells wrap your message in order. The motion does not say that.

Each of the six scene objects — `Core`, `ScatterCloud`, `LockShell`, `SignatureRing`, `Lattice`, `AnchorPulse` — receives a **boolean**:

```
<Core         active={activeId === id(0)} reduced={reducedMotion} />
<ScatterCloud active={activeId === id(1)} reduced={reducedMotion} />
<LockShell    active={activeId === id(2)} reduced={reducedMotion} />
```

Each then damps toward that boolean independently inside its own `useFrame`:

```
const approach = (current, goal, delta, rate = 0.0025) =>
  current + (goal - current) * (1 - Math.pow(rate, delta));
```

This is good code. `approach()` is frame-rate-independent exponential damping and it retargets for free when `active` flips mid-transition. **It is not the problem and should not be replaced.**

The problem is that a boolean carries no timing information. Six objects each racing toward their own target, started at the same instant, can only ever read as "everything shifts at once." There is no shared clock, so there is no way to express *shell 5 opens, then 4, then 3*.

That ordering is the one thing the section exists to communicate, and it is structurally impossible in the current architecture.

2. The change in one sentence

Replace the binary `active` prop with a **numeric reveal value (0 to 1) owned by a single GSAP timeline**, which the frame loop reads.

This keeps every existing motion signature intact. Each object still moves the way it moves. The only difference is that a timeline now decides *when* each one starts.

Why a proxy object, not direct tweening

Do **not** tween three.js objects with GSAP directly. `useFrame` writes to the same properties every frame and the two will fight, producing stutter that looks like a performance bug.

The correct pattern is a plain JavaScript object that GSAP owns and the frame loop only reads:

```
GSAP timeline  --writes-->  reveal[] (plain numbers)  --reads-->  useFrame
```

GSAP never touches the scene graph. The frame loop remains the only writer.

---

3. File changes

3.1 LayerScene.tsx — add the reveal state to Rig

`Rig` (line 253) becomes the owner of the timeline and the shared numeric state.

```
import { useGSAP } from "@gsap/react";
import { gsap } from "gsap";

function Rig({ activeId, reducedMotion }: SceneProps) {
  const group = useRef<THREE.Group>(null);

  // Owned by GSAP, read by every child's useFrame. Never written in a frame.
  const reveal = useRef([0, 0, 0, 0, 0, 0]);

  useGSAP(
    () => {
      const activeIndex = activeId
        ? LAYERS.findIndex((l) => l.id === activeId)
        : -1;

      // Reduced motion: jump, do not animate. Matches the existing contract.
      if (reducedMotion) {
        reveal.current = reveal.current.map((_, i) =>
          i === activeIndex ? 1 : 0,
        );
        return;
      }

      const tl = gsap.timeline({ defaults: { ease: "expo.out" } });

      // Retract everything not selected, fast and together.
      reveal.current.forEach((_, i) => {
```

```
        if (i === activeIndex) return;
        tl.to(reveal.current, { [i]: 0, duration: 0.28 }, 0);
      });

      // Reveal the selected shell, delayed by its depth so the ripple
      // travels outward from the core.
      if (activeIndex >= 0) {
        tl.to(
          reveal.current,
          { [activeIndex]: 1, duration: 0.5 },
          activeIndex * 0.06,
        );
      }
    },
    { dependencies: [activeId, reducedMotion] },
  );

  // ...existing rotation useFrame unchanged...
}
```

`useGSAP` reverts the timeline automatically when `activeId` changes or the component unmounts. That is what prevents the overlapping-tween bug you would otherwise hit clicking layers quickly.

3.2 Children — accept a number instead of a boolean

Each of the six components changes its signature. Using `Core` as the worked example:

```
// before
function Core({ active, reduced }: { active: boolean; reduced: boolean }) {
  useFrame((state, delta) => {
    mesh.current.scale.setScalar(breathe * (active ? 1.25 : 1));
    mat.current.emissiveIntensity = approach(
      mat.current.emissiveIntensity, active ? 5.5 : 3, delta,
    );
  });
}

// after
function Core({ getReveal, reduced }: { getReveal: () => number; reduced: boolean }) {
  useFrame((state, delta) => {
    const r = getReveal();                       // 0 -> 1, driven by GSAP
    mesh.current.scale.setScalar(breathe * THREE.MathUtils.lerp(1, 1.25, r));
    mat.current.emissiveIntensity = approach(
      mat.current.emissiveIntensity, THREE.MathUtils.lerp(3, 5.5, r), delta,
    );
  });
}
```

The rule is mechanical: **every `active ? A : B` becomes `THREE.MathUtils.lerp(B, A, r)`.**

Keep the inner `approach()` calls. GSAP now controls the macro timing; `approach()` still smooths the per-frame response. The two compose correctly because they operate at different levels.

Pass the getter, not the value, so React does not re-render on every frame:

```
<Core          getReveal={() => reveal.current[0]} reduced={reducedMotion} />
<ScatterCloud  getReveal={() => reveal.current[1]} reduced={reducedMotion} />
<LockShell     getReveal={() => reveal.current[2]} reduced={reducedMotion} />
<SignatureRing getReveal={() => reveal.current[3]} reduced={reducedMotion} />
<Lattice       getReveal={() => reveal.current[4]} reduced={reducedMotion} />
<AnchorPulse   getReveal={() => reveal.current[5]} reduced={reducedMotion} />
```

3.3 EncryptionLayers.tsx — join the detail panel to the same timeline

Currently the panel uses `key={active.id}` with a `fade-in-up` class. That is a hard remount: the old copy vanishes on frame one, and there is no exit at all.

Replace with a GSAP crossfade so the panel lands *after* the shell it describes:

Phase	Values
Out (old copy)	`opacity: 0, y: -8, duration: 0.18, ease: power2.in`
In (new copy)	`opacity: 0 to 1, y: 12 to 0, duration: 0.32, ease: expo.out`

Position	starts at `-=0.25` relative to the scene reveal

Keep `min-h-[132px]` on the container. It is what prevents the layout jump during the swap, and removing it will reintroduce one.

4. Timing specification

Element	Delay	Duration	Ease
Deselect non-active shells	0	0.28s	`expo.out`
Selected shell reveal	`index * 0.06s`	0.5s	`expo.out`
Detail panel out	0	0.18s	`power2.in`
Detail panel in	scene `-0.25s`	0.32s	`expo.out`

Total perceived transition for layer 5: roughly **0.80s**. This sits in the marketing and explanatory budget, not the sub-300ms UI budget, which is correct — this motion is the explanation, not interface feedback.

The easing trap

GSAP silently ignores CSS easing strings. This has already cost time on this machine once.

```
gsap.to(obj, { ease: "cubic-bezier(0.16, 1, 0.3, 1)" })  // no error, uses the default
```

The project's `--ease-out` token is `cubic-bezier(0.16, 1, 0.3, 1)`, which is very close to GSAP's built-in `expo.out`. Use `expo.out`. If exact parity with the CSS token matters, register `CustomEase` explicitly — but do not pass the token string and assume it took effect.

---

5. Verification

A green build proves nothing here. A parameter can be accepted and ignored.

1. **Watch it, do not read it.** Click each of the six layers and confirm the reveal ripples outward from the core rather than everything moving at once.
2. **Interrupt it.** Click layers 0, then 5, then 2 in rapid succession. Confirm no stacked or fighting tweens, and that the panel does not flash a stale value. This is the failure mode `useGSAP` cleanup exists to prevent.
3. **Assert on the effect, not the call.** Log `reveal.current` during a transition and confirm the numbers move through intermediate values. If they snap 0 to 1, the ease was dropped.
4. **Reduced motion.** Set `prefers-reduced-motion: reduce` and confirm the scene jumps to final state with no tween, matching the existing contract in `usePrefersReducedMotion`.
5. **Frame budget.** The frame loop is unchanged in cost — GSAP writes six numbers per tick, not per frame. If frame time regresses, something is tweening the scene graph directly, which section 2 forbids.

---

6. Risks

Risk	Mitigation
GSAP and `useFrame` writing the same property	Enforce the proxy pattern. GSAP writes only to `reveal.current`.
Re-render per frame from passing values as props	Pass getters (`getReveal`), not numbers.

Overlapping timelines on rapid clicks	`useGSAP` with `dependencies: [activeId]` reverts the previous timeline.
Easing token silently dropped	Use `expo.out`, never a `cubic-bezier(...)` string.
Scope creep into the frame loop	`approach()` stays. This plan changes *when* things start, not *how* they move.

---

7. What this plan deliberately excludes

- **Replacing `useFrame` damping with GSAP.** The damping is correct and retargets for free. Replacing it would add cost and lose interrupt behaviour.
- **Scroll-scrubbed layer progression.** It hijacks scroll on a section whose real interaction is clicking, and traps readers who want to scroll past.
- **Per-character text animation on the body paragraph.** Explanatory copy people are reading; character stagger delays comprehension.
- **Button and chevron transitions.** Already correct at `duration-200`.